Fiche de TD n° 01
Complexité
(Exercices Supplémentaires)

Exercice n°01 : Obtenez la complexité temporelle du programme suivant :

```
def whirlpool(m):
    if m <= 0:
        print("eddy!")
        return
    else:
        whirlpool(m - 1)
        whirlpool(m - 1)
        print("whirl")
whirlpool(3)  # Exemple d'appel de la fonction avec m = 3
```

Exercice n°02 : Combien de fois la ligne 3 est atteinte et traitée pendant l'exécution dans le fragment de programme suivant est ___.

a) $\frac{4n^2-2n}{2}$ b) $\frac{i^2-i}{2}$ c) $\frac{(i^2-3i)}{2}$ d) $\frac{(4n^2-6n)}{2}$

```
1. i = 2 * n
2. for j in range(1, i + 1):
3.  for k in range(3, j + 1):
4.  m = m + 1
```

Exercice n° 03:
Trouver la complexité temporelle du fragment de programme suivant :

```
1.for i in range(20, 30):
2.  for j in range(1, n + 1):
3.      am = am + 1
```

Exercice n° 04: Indiquez l'affirmation suivante qu'elle soit vraie ou fausse.

$f(n) = 30n^2 2^n + 6n2^n + 8n^2 = O(2^n)$

Exercice n° 05 : Utilisez la notation **Grand O** pour décrire la complexité temporelle de la fonction suivante qui détermine si une année donnée est bissextile ou non:

```
def EstBissextile (annee):
    return (annee % 100 == 0) and (annee % 400 == 0) or (annee % 4 == 0)
```

Exercice n° 06 : Utilisez la notation **Grand O** pour décrire la complexité temporelle de la fonction suivante qui additionne tous les nombres d'un tableau donné :

```
def arraySum(array):
    sum = 0
    for i in range(len(array)):
        sum += array[i]
```

```
    return sum
```

Exercice n° 07 : La fonction suivante est basée sur l'analogie ancestrale utilisée pour décrire le pouvoir de la croissance des intérêts composés :

Imaginez que vous ayez un échiquier et que vous placiez un seul grain de riz sur une case. Sur la deuxième case, vous placez 2 grains de riz, car c'est le double de la quantité de riz sur la case précédente. Sur la troisième case, vous placez 4 grains. Sur la quatrième case, vous placez 8 grains, et sur la cinquième case, vous placez 16 grains, et ainsi de suite. La fonction suivante calcule sur quelle case vous devrez placer un certain nombre de grains de riz. Par exemple, pour 16 grains, la fonction renverra 5, car vous placerez les 16 grains sur la cinquième case.

Utilisez la notation **Grand O** pour décrire la complexité temporelle de cette fonction, qui est la suivante :

```
def chessboardSpace(numberOfGrains):
    chessboardSpaces = 1
    placedGrains = 1
    while placedGrains < numberOfGrains:
        placedGrains *= 2
        chessboardSpaces += 1
    return chessboardSpaces
```

Exercice n° 08:

Remplacez les points d'interrogation dans le tableau suivant pour décrire combien d'étapes se produisent pour un nombre donné d'éléments de données à travers divers types de notation **Grand O** :

N Elements	O(N)	O(log N)	O(N^2)
100	100	**?**	**?**
2000	**?**	**?**	**?**

Exercice n° 09 : Si nous avons un algorithme en O(n^2) qui traite un tableau et que nous constatons qu'il prend 256 étapes, quelle est la taille du tableau ?

Exercice n° 10: Utilisez la notation **Grand O** pour décrire la complexité temporelle de la fonction suivante. Elle trouve le produit le plus élevé parmi toutes les paires de deux nombres dans un tableau donné :

```
def greatestProduct(array):
    greatestProductSoFar = array[0] * array[1]
    for i, iVal in enumerate(array):
        for j, jVal in enumerate(array):
            if i != j and iVal * jVal > greatestProductSoFar:
                greatestProductSoFar = iVal * jVal
    return greatestProductSoFar
```

Exercice n° 11 : La fonction suivante trouve le plus grand nombre unique dans un tableau, mais elle a une efficacité de **O(n^2).** Réécrivez la fonction pour qu'elle devienne plus rapide avec une efficacité de **O(n).**

```
def greatestNumber(array):
    for i in array:
        # Supposons pour le moment que i est le plus grand :
        isIValTheGreatest = True
        for j in array:
            # Si nous trouvons une autre valeur qui est plus grande que i,
            # i n'est pas le plus grand :
            if j > i:
                isIValTheGreatest = False
        # Si, à la fin de la vérification de tous les autres nombres, i
        # est toujours le plus grand, cela signifie qu'i est le plus grand nombre :
        if isIValTheGreatest:
            return i
```

Exercice n° 12 : Utilisez la notation **Grand O** pour décrire la complexité temporelle d'un algorithme qui effectue 4n + 16 étapes.

Exercice n° 13 : Utilisez la notation **Grand O** pour décrire la complexité temporelle d'un algorithme qui effectue $2n^2$

Exercice n° 14:

Utilisez la notation **Grand O** pour décrire la complexité temporelle de la fonction suivante.

Explication : Cette fonction résout un problème célèbre connu sous le nom de "**recherche d'une aiguille dans une botte de foin**". Tant l'aiguille (needle الإبرة) que la botte de foin (haystack كومة قش) sont des chaînes de caractères. Par exemple, si l'aiguille est "def" et la botte de foin est "abcdefghi", l'aiguille se trouve quelque part dans la botte de foin, car "def" est une sous-chaîne de "abcdefghi". Cependant, si l'aiguille est "dd", elle ne peut pas être trouvée dans la botte de foin "abcdefghi". Cette fonction renvoie vrai ou faux, en fonction de la présence ou de l'absence de l'aiguille dans la botte de foin :

```
def find_needle(needle, haystack):
    needle_index = 0
    haystack_index = 0
    while haystack_index < len(haystack):
        if needle[needle_index] == haystack[haystack_index]:
            found_needle = True
            while needle_index < len(needle):
                if needle[needle_index] != haystack[haystack_index + needle_index]:
                    found_needle = False
                    break
                needle_index += 1
            if found_needle:
                return True
```

```
            needle_index = 0
        haystack_index += 1
    return False
```